

Programming Assignment: Unit 4

Hset Paing Htoo

Introduction to Computer Science Program, University of the People

CS 1102-01 Programming 1 AY2026-T5

Instructor : Sirajo Musa (Instructor)

July 16, 2026

Stock Price Analysis: Array and ArrayList Processing in Java

1. Overview

This assignment processes ten days of stock opening prices to calculate summary statistics using Java arrays and ArrayLists. Four methods were implemented: `calculateAveragePrice` and `findMaximumPrice` and `countOccurrences` operate on a float array, while `computeCumulativeSum` operates on an `ArrayList<Float>` and returns a new `ArrayList` holding the running total at each position. Working with both structures side by side highlights their key differences: an array has a fixed length set at creation and is accessed with the `[]` operator, while an `ArrayList` can grow dynamically and is accessed through `get()` and `add()` methods (Eck, 2022, Sections 7.1 and 7.3).

2. Program Code

The complete program, `StockPriceAnalyzer.java`, is shown below.

```
import java.util.ArrayList;

/**
 * StockPriceAnalyzer
 *
 * Demonstrates array and ArrayList processing in Java by calculating
 * the average, maximum, occurrence count, and cumulative sum of a
 * 10-day set of stock opening prices.
 */
public class StockPriceAnalyzer {

    /**
     * Calculates the average of all prices in the array.
     * @param prices array of stock prices
     * @return the average price
     */
    public static float calculateAveragePrice(float[] prices) {
        float sum = 0;
        for (int i = 0; i < prices.length; i++) {
```

```

        sum += prices[i];
    }
    return sum / prices.length;
}

/**
 * Finds the highest price in the array.
 * @param prices array of stock prices
 * @return the maximum price
 */
public static float findMaximumPrice(float[] prices) {
    float max = prices[0];
    for (int i = 1; i < prices.length; i++) {
        if (prices[i] > max) {
            max = prices[i];
        }
    }
    return max;
}

/**
 * Counts how many times a specific target price appears in the array.
 * @param prices array of stock prices
 * @param targetPrice the price to search for
 * @return the number of occurrences of targetPrice
 */
public static int countOccurrences(float[] prices, float targetPrice) {
    int count = 0;
    for (int i = 0; i < prices.length; i++) {
        if (prices[i] == targetPrice) {
            count++;
        }
    }
    return count;
}

/**
 * Computes a running (cumulative) sum of the prices in an ArrayList.
 * @param prices ArrayList of stock prices
 * @return a new ArrayList where each position holds the sum of all
 *         prices up to and including that position
 */
public static ArrayList<Float> computeCumulativeSum(ArrayList<Float> prices) {
    ArrayList<Float> cumulativeSum = new ArrayList<Float>();
    float runningTotal = 0;
    for (int i = 0; i < prices.size(); i++) {
        runningTotal += prices.get(i);
        cumulativeSum.add(runningTotal);
    }
    return cumulativeSum;
}

```

```

public static void main(String[] args) {
    // 10 days of opening stock prices, stored as a float array
    float[] stockPrices = {102.5f, 105.0f, 101.75f, 105.0f, 108.25f,
                           110.0f, 107.5f, 105.0f, 109.75f, 111.0f};

    // The same data stored as an ArrayList, used for the cumulative sum task
    ArrayList<Float> stockPriceList = new ArrayList<Float>();
    for (int i = 0; i < stockPrices.length; i++) {
        stockPriceList.add(stockPrices[i]);
    }

    // Task 1: average price
    float averagePrice = calculateAveragePrice(stockPrices);
    System.out.println("Average stock price: " + averagePrice);

    // Task 2: maximum price
    float maximumPrice = findMaximumPrice(stockPrices);
    System.out.println("Maximum stock price: " + maximumPrice);

    // Task 3: occurrences of a target price
    float targetPrice = 105.0f;
    int occurrences = countOccurrences(stockPrices, targetPrice);
    System.out.println("Occurrences of " + targetPrice + ": " + occurrences);

    // Task 4: cumulative sum
    ArrayList<Float> cumulativeSum = computeCumulativeSum(stockPriceList);
    System.out.println("Cumulative sum: " + cumulativeSum);
}
}

```

3. Explanation of the Code

3.1 calculateAveragePrice(float[] prices)

This method loops through the array with a standard for loop, accumulating a running sum, then divides that sum by prices.length once the loop finishes. Because an array stores its size in the built-in length field rather than a method call, this is a direct field access (Eck, 2022, Section 7.1).

3.2 findMaximumPrice(float[] prices)

This method uses the common “running maximum” pattern: the first array element seeds the max variable, and every subsequent element is compared against it, updating max whenever a larger value is found. This pattern is discussed in the array-processing material in Eck (2022, Section 7.2).

3.3 countOccurrences(float[] prices, float targetPrice)

This method performs a linear search across the array, incrementing a counter whenever an element equals targetPrice. Comparing float values with == is safe here because the array holds fixed literal values with no accumulated rounding error; with prices computed at runtime, a small tolerance comparison would be safer.

3.4 computeCumulativeSum(ArrayList<Float> prices)

This method builds a new ArrayList<Float>, using get(i) and size() — the ArrayList equivalents of array indexing and the length field — to add each price to a running total and store that total at the matching position in the result list. This mirrors the ArrayList usage described in Eck (2022, Section 7.3).

3.5 Passing Arrays and ArrayLists to Methods

All four methods receive their array or ArrayList as a parameter without copying the underlying data, because arrays and ArrayLists are objects in Java and are passed by passing a reference to that object. None of the methods in this program modify the caller's data — each one only reads values or returns a new object — but this reference-passing behavior means that if a method did reassign elements inside the array or list, those changes would be visible to the caller after the method returns (Eck, 2022, Sections 4.3.5 and 5.1.4).

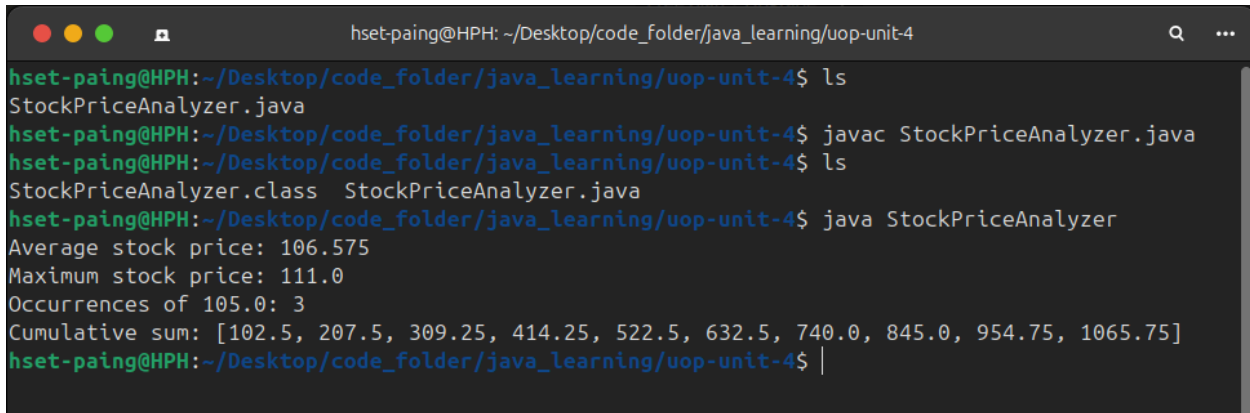
4. Sample Data and Expected Output

Using the ten sample prices declared in main — 102.5, 105.0, 101.75, 105.0, 108.25, 110.0, 107.5, 105.0, 109.75, and 111.0 — the program produces the following results:

- Average stock price: 106.575
- Maximum stock price: 111.0
- Occurrences of 105.0: 3
- Cumulative sum: [102.5, 207.5, 309.25, 414.25, 522.5, 632.5, 740.0, 845.0, 954.75, 1065.75]

5. Program Output

Screenshot of the compiled program running in the console:



```
hset-paing@HPH: ~/Desktop/code_folder/java_learning/uop-unit-4
hset-paing@HPH:~/Desktop/code_folder/java_learning/uop-unit-4$ ls
StockPriceAnalyzer.java
hset-paing@HPH:~/Desktop/code_folder/java_learning/uop-unit-4$ javac StockPriceAnalyzer.java
hset-paing@HPH:~/Desktop/code_folder/java_learning/uop-unit-4$ ls
StockPriceAnalyzer.class  StockPriceAnalyzer.java
hset-paing@HPH:~/Desktop/code_folder/java_learning/uop-unit-4$ java StockPriceAnalyzer
Average stock price: 106.575
Maximum stock price: 111.0
Occurrences of 105.0: 3
Cumulative sum: [102.5, 207.5, 309.25, 414.25, 522.5, 632.5, 740.0, 845.0, 954.75, 1065.75]
hset-paing@HPH:~/Desktop/code_folder/java_learning/uop-unit-4$ |
```

References

Eck, D. J. (2022). *Introduction to programming using Java, version 9, JavaFX edition* (Licensed under CC 4.0). <https://math.hws.edu/javanotes/>